Design, Bit-Exact Verification, and PPA Characterization of a 256-Point R2SDF FFT Audio Spectrum Analyzer IP Core on Zynq-7000

Anh Q. Vu, Hung D. Truong, Hoang H.N. Le,
Minh D. Nguyen, Duyen K.V. Nguyen, Loc D. Lam
FPT University, Ho Chi Minh Campus, Ho Chi Minh City, Vietnam
anhvuvlsi@gmail.com, donghung1501@gmail.com, lenguyenhuyhoang13112006@gmail.com,
duyminhnguyen2020@gmail.com, nguyenvukyduyen2911@gmail.com, loc.slh11357@gmail.com

***Abstract*—This paper reports the design, layered functional verification, and power–performance–area (PPA) characterization of a 256-point radix-2 single-path delay feedback (R2SDF) FFT audio spectrum analyzer IP core, written in Verilog and implemented on a Xilinx XC7Z020 (PYNQ-Z2). The datapath uses Q1.15 arithmetic with a one-half scaling schedule per stage, a three-multiplier complex product, a Hann analysis window, an equiripple α-max-β-min magnitude estimator, and AXI4-Stream framing. Every block is verified against a bit-accurate Python reference model: 200 000 random vectors for the complex multiplier, an exhaustive twiddle-ROM comparison, cycle-accurate comparison of the butterfly and of the bit-reversal output buffer, and full-frame comparison of the FFT core — all bit-exact. We isolate and document an error class that value-level golden models are structurally unable to detect: an eight-cycle framing error, produced by modelling the pipeline in the wrong time domain, that leaves the arithmetic bit-exact while corrupting every reported bin (peak bin 122 instead of 10). Correcting the latency constant from 283 to 291 valid samples — derived here analytically as 255 delay-line $+\,8$ butterfly output-register $+\,28$ twiddle-branch cycles — restores exact agreement. On XC7Z020 the complete board build occupies 8.33 % of the LUTs, 4.07 % of the flip-flops, 5.36 % of the block RAM and 10.91 % of the DSP slices (24 DSP48E1, matching an analytic resource model exactly); characterised out of context, the IP alone occupies 6 %, 2 % and 3 % of the same resources at an unchanged DSP count. Timing closes with $+2.510$ ns worst negative slack at 49.98 MHz on the board, where the peak-bin self-test is confirmed on silicon, and the binding-constraint $F_{\max}$ is ≈ 64 MHz, i.e. $\approx 1330\times$ the 48 kS/s real-time requirement. We quantify the cost of the R2SDF choice against an R2^{2}SDF baseline (7 versus 3 complex multipliers) and state explicitly which results are measured, which are derived, and which remain open.**

***Index Terms*— AXI4-Stream, bit-exact verification, fast Fourier transform (FFT), field-programmable gate array (FPGA), fixed-point arithmetic, pipeline latency, power–performance–area (PPA), radix-2 single-path delay feedback (R2SDF), spectrum analysis, timing closure.**

I. Introduction

Real-time spectral analysis is the front end of a large class of embedded audio systems: keyword spotting, acoustic event detection, active noise control, machine condition monitoring, and instrumentation. In field-programmable gate array (FPGA) designs the fast Fourier transform (FFT) that performs this analysis is usually instantiated from a vendor library, which is the correct engineering decision when the transform is a fixed commodity. It is the wrong decision when the arithmetic itself is the object of study — for example when precision must be scaled against a downstream task metric, when only a few bins are required, or when a course or a research group needs a transform whose every rounding decision is visible and verifiable.

This paper describes such a core: a 256-point streaming FFT audio spectrum analyzer, built from primitives, and — more to the point — verified against a bit-accurate software reference at every level of its hierarchy. The architecture is deliberately conventional. The Cooley–Tukey decomposition [1], the single-path delay-feedback (SDF) pipeline [5, 6], Q1.15 fixed-point arithmetic with per-stage scaling [2, 10], the three-multiplier complex product [12,13], the Hann window [14], and the α-max-β-min magnitude estimator [15,16] are all textbook. We make no claim of architectural novelty, and Section VI D quantifies what our architectural choice costs relative to the accepted baseline.

The contribution is instead quantitative and methodological, and it rests on an observation that emerged repeatedly during development: in a pipelined, valid-gated streaming FFT, the difficult part is not the arithmetic. The arithmetic was bit-exact against the reference model from an early stage. The difficult part is the relationship between a numerically correct datapath and the control and framing signals that label its output. A datapath can be exactly right and the system still wrong, and the failure is invisible to the very reference model that was built to catch it.

Contributions

1. A complete, openly available RTL description of a 256-point R2SDF FFT spectrum analyzer with AXI4-Stream framing, together with the bit-accurate Python model that generates its coefficient memories and serves as its verification oracle (Sections III and IV).

2. A layered verification result: seven verification levels, from the complex multiplier to the complete system, all bit-exact, with the quantitative evidence for each level tabulated (Table IV).

3. An analysis of a pipeline-framing error class that value-level golden models cannot detect, including the analytic latency decomposition that resolves it ($291 = 255 + 8 + 28$ valid samples), the reason the original testbench could not observe it, and the design rules that prevent it (Sections V B and VII).

4. A PPA characterization on XC7Z020 with an analytic resource model that predicts the measured DSP count exactly, a three-point timing study showing how strongly reported $F_{\max}$ depends on the constraint under which the tool was run, and an explicit statement of which numbers are not yet publication-grade (Section VI).

5. An independent re-derivation, performed for this paper with a separately written bit-accurate model, of the core's fixed-point behaviour: 6.02 dB/bit SQNR slope, 58.97 dB at Q1.15 against a double-precision reference, and the same peak bin as the hardware (Section IV E).

The remainder of the paper is organised as follows. Section II develops the theoretical background. Section III describes the architecture and implementation. Section IV presents the verification methodology. Section V reports functional and timing results and the key findings. Section VI gives the PPA characterization. Section VII generalises the lessons into design rules, Section VIII states the limitations, and Section IX concludes.

II. Theoretical Background

A. Discrete Fourier Transform and the Radix-2 DIF Decomposition

For a length-N complex sequence $x[n]$ the discrete Fourier transform (DFT) is

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{nk}, \qquad W_N = e^{-j2\pi/N}, \quad 0 \le k < N. \tag{1}$$

Direct evaluation costs $\mathcal{O}(N^2)$ complex multiplications. The Cooley–Tukey algorithm [1] factors (1) recursively; in the decimation-in-frequency (DIF) form the even and odd output bins separate as

$$X[2r] = \sum_{n=0}^{N/2-1} \big(x[n] + x[n+N/2]\big)\, W_{N/2}^{nr}, \tag{2}$$

$$X[2r+1] = \sum_{n=0}^{N/2-1} \big(x[n] - x[n+N/2]\big)\, W_N^{n} W_{N/2}^{nr}, \tag{3}$$

with $0 \le r < N/2$. Each recursion level performs $N/2$ butterflies, one complex multiplication by a *twiddle factor* W_N^n per butterfly, and halves the transform length. For $N = 2^S$ the recursion terminates after S levels at a cost of $\mathcal{O}(N \log_2 N)$ operations [3]. The present design uses $N = 256$, hence $S = 8$ stages. Two structural properties of (2)–(3) drive the hardware: the twiddle multiplication is required only on the difference branch, and the natural-order input produces a bit-reversed output ordering.

TABLE I
Pipelined FFT architectures for $N = 256$, one sample per cycle

Architecture	Non-trivial complex mult.	Complex adders	Memory (cplx words)
R2SDF (this work)	$\log_2 N - 1 = 7$	$2\log_2 N = 16$	$N - 1 = 255$
R4SDF	$\log_4 N - 1 = 3$	$8\log_4 N = 32$	$N - 1 = 255$
R2^2SDF	$\log_4 N - 1 = 3$	$2\log_2 N = 16$	$N - 1 = 255$

B. From Dataflow Graph to Single-Path Delay Feedback Pipeline

A pipelined FFT maps one hardware stage to each recursion level and streams samples through it, trading the N-fold parallelism of the dataflow graph for a throughput of one sample per clock [5,9]. In the SDF family [6, 7], stage s ($0 \le s < S$) owns a feedback shift register of

$$L_s = \frac{N}{2^{s+1}} \tag{4}$$

complex words. During the first L_s valid samples of a block the input is written into the register; during the next L_s samples the butterfly consumes the register output a and the current input b and produces $(a+b)$ on the forward path while $(a-b)$, multiplied by the appropriate twiddle factor, is pushed back into the register. A single control bit per stage — the commutator select — alternates between these phases. The total delay-line requirement is

$$\sum_{s=0}^{S-1} \frac{N}{2^{s+1}} = N - 1, \tag{5}$$

which for $N = 256$ is 255 complex words: SDF attains the theoretical minimum memory for a pipelined FFT, and it is this property, together with 100 % multiplier utilisation, that makes the family attractive for area-constrained streaming applications [9].

The variants of the family differ mainly in how many of the twiddle rotations are non-trivial. Plain R2SDF requires one general complex multiplier in every stage except the last, i.e. $\log_2 N - 1$ multipliers. Radix-2^2 SDF (R2^2SDF) [7] rewrites the same graph so that every second rotation degenerates to a multiplication by $-j$, which is a swap and a sign inversion, leaving only $\log_4 N - 1$ general multipliers while retaining the radix-2 butterfly and the $N - 1$ memory. Feedforward (multi-path delay commutator) architectures raise throughput by processing several samples per cycle at proportionally higher memory cost [8]. Table I summarises the comparison for $N = 256$; the consequences for this design are quantified in Section VI D.

C. Fixed-Point Arithmetic, Scaling, and SQNR

All data are held in Q1.15: a 16-bit two's-complement word interpreted as a fraction in $[-1,\, 1 - 2^{-15}]$ with quantisation step $\varepsilon = 2^{-15}$. The butterfly sum and difference of two operands of magnitude up to unity can reach magnitude two, so the design applies a *scaling schedule* of one right shift per stage,

$$y_{\pm} = \left\lfloor \frac{a \pm b}{2} \right\rceil_{\text{rne}}, \tag{6}$$

where $\lfloor\cdot\rceil_{\mathrm{rne}}$ denotes round-half-to-even followed by saturation. Because $|a \pm b|/2 \le \max(|a|,|b|)$, the schedule makes overflow structurally impossible rather than merely unlikely. Its effect is not marginal: with the schedule removed, the measured output SQNR of this core collapses from ≈ 60 dB to ≈ 1.5 dB, overflow wrap-around rather than quantisation becoming the dominant error term. The accumulated $2^{-S} = 1/N$ gain means the core computes $X[k]/N$, the convention required for a windowed magnitude display. Round-half-to-even is used instead of round-half-up because the latter introduces a positive mean error of $\varepsilon/2$ per operation, which in an eight-stage pipeline accumulates into a visible DC bias.

Each rounding operation injects an error that is well modelled as uniform white noise of variance $\sigma_e^2 = \varepsilon^2/12$ [2,10,11]. Under the one-half-per-stage schedule, noise injected in stage m is attenuated by the same factor as the signal that follows it, so the noise contributions of successive stages add with equal weight while the signal is divided by N. The classical first-order result is that the output noise-to-signal ratio grows proportionally to N, i.e. the design loses about 3 dB — half a bit — per stage. Writing the full-scale sinusoidal signal-to-quantisation noise ratio of a B-bit converter as $6.02(B-1)+1.76$ dB, the expected core output SQNR is

$$\mathrm{SQNR} \;\approx\; 6.02\,(B-1) + 1.76 - 3.01\log_2 N \;\;[\mathrm{dB}]. \tag{7}$$

For $B = 16$ and $N = 256$ this gives 68.0 dB. Equation (7) counts butterfly rounding only; twiddle-coefficient quantisation and the rounding inside each complex multiplier lower it further, and Section IV E measures the residual gap.

D. Three-Multiplier Complex Product

A complex product $(a_r + ja_i)(b_r + jb_i)$ requires four real multiplications in its direct form. The classical three-multiplication identity — attributed to Gauss and structurally identical to the Karatsuba scheme for integers [12, 13] — computes

$$\begin{aligned} &k_1 = b_r\,(a_r + a_i), \quad k_2 = a_r\,(b_i - b_r), \quad k_3 = a_i\,(b_r + b_i), \\ &p_r = k_1 - k_3, \qquad\qquad p_i = k_1 + k_2, \end{aligned} \tag{8}$$

trading one multiplication for two extra additions. On a 7-series FPGA a 16×16 signed multiplication maps to one DSP48E1 [23], so the identity saves one DSP slice per complex multiplier, i.e. seven slices in this design. Because (8) is an exact integer identity, it must produce bit-identical results to the direct form provided rounding is applied only once, after the final addition — a property we confirm empirically in Section IV. The intermediate sums require one guard bit, and the products are formed in Q2.30 before a single round-and-saturate back to Q1.15.

E. Windowing

Because the analyzer observes a finite 256-sample record of a continuous signal, any component that is not exactly bin-centred leaks into neighbouring bins. Multiplying the record by a Hann window

$$w[n] = \tfrac{1}{2}\Big(1 - \cos\tfrac{2\pi n}{N}\Big), \qquad 0 \le n < N, \tag{9}$$

suppresses the highest sidelobe to -31.5 dB and imposes an asymptotic roll-off of -18 dB/octave, against -13.3 dB and -6 dB/octave for the rectangular window [14]. The price is a wider main lobe: the Hann window has coherent gain 0.5, equivalent noise bandwidth (ENBW) 1.5 bins, maximum scalloping loss 1.42 dB and worst-case processing loss 3.18 dB, all of which we reproduce numerically for the $N = 256$ periodic window used here. A useful consequence for verification is that the Hann window has the exact three-tap frequency-domain kernel $[-\frac{1}{4}, \frac{1}{2}, -\frac{1}{4}]$: a tone placed exactly on bin k_0 therefore produces non-zero energy in exactly three bins, with the two neighbours precisely 6.02 dB below the peak. Any deviation from that pattern in a self-test is diagnostic. The window coefficients are stored in a 256×16-bit ROM in Q1.15 and applied with the same round-and-saturate rule as the datapath.

F. Magnitude Estimation

The display and peak-picking stages need $|X[k]|$, not the complex value. An exact magnitude requires a square-root; the α-max-β-min estimator

$$|z| \;\approx\; \alpha \max\big(|z_r|,|z_i|\big) + \beta \min\big(|z_r|,|z_i|\big) \tag{10}$$

replaces it with two multiplications and one addition [15, 16]. The coefficient pair that minimises the peak error is $\alpha = 0.960434$, $\beta = 0.397825$; quantised to Q1.15 the implementation stores $\alpha = 31471/2^{15}$ and $\beta = 13036/2^{15}$, which we verify to give an equiripple error of $\pm 3.96\,\%$ (0.34 dB), against 11.8 % for the popular $(\alpha,\beta) = (1, \frac{1}{2})$ pair (Fig. 6b). The error is a smooth function of phase angle only, so it neither biases the spectrum in frequency nor changes the relative ordering of bins that differ by more than 0.34 dB — a sufficient guarantee for peak detection, and the reason the estimator is acceptable here even though it would not be in a measurement instrument. A leading-one detector over 16 levels followed by mantissa interpolation converts the linear magnitude to $\log_2$ for decibel-scaled display.

G. Output Ordering and Bin Mapping

The DIF recursion consumes samples in natural order and emits them in bit-reversed order: output index n carries bin $k = \mathrm{bitrev}_S(n)$. Reordering is performed by writing the stream into a dual-port RAM at bit-reversed addresses and reading it linearly. For a real input the spectrum is Hermitian, $X[N-k] = X^*[k]$, so only the $N/2 = 128$ single-sided bins are retained. Bin k corresponds to

$$f_k = \frac{k f_s}{N}, \tag{11}$$

which at $f_s = 48$ kHz and $N = 256$ gives a resolution of 187.5 Hz and places the on-board self-test tone, bin 10, at 1875 Hz.

III. Architecture and Implementation

A. System Overview

Fig. 1 shows the processing chain and Table II the design parameters. The IP is packaged behind AXI4-Stream slave and

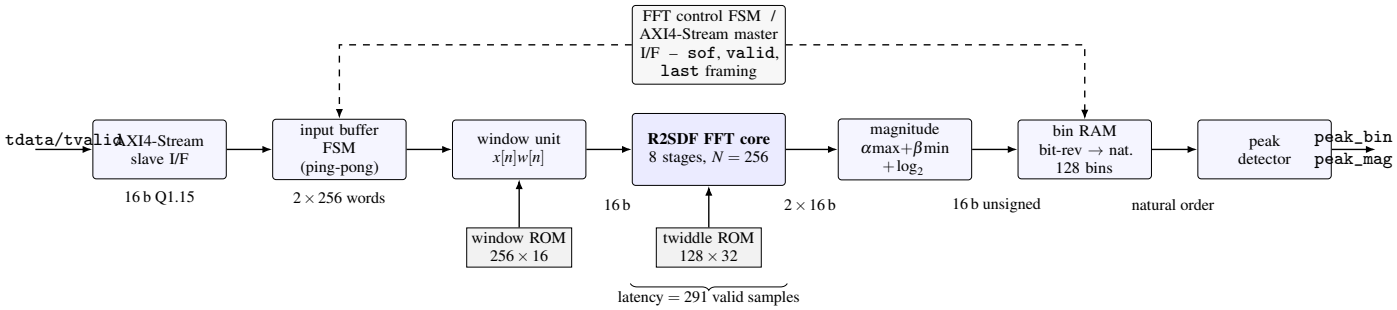

Fig. 1. Processing chain of the spectrum analyzer IP. Audio samples enter over AXI4-Stream, are buffered into ping-pong frames of 256 samples, multiplied by a Hann window, transformed by the eight-stage R2SDF pipeline, converted to magnitude, reordered from bit-reversed to natural bin order, and reduced to a per-frame peak. Framing signals (`sof`, `valid`, `last`) are propagated through delay lines matched to the datapath latency rather than regenerated, which is the mechanism that made the latency constant safety-critical (Section V B).

TABLE II
DESIGN PARAMETERS

Parameter	Value
Transform length N	256 (8 radix-2 DIF stages)
Architecture	R2SDF, decimation in frequency
Data format	Q1.15 signed, real and imaginary
Scaling schedule	$1/2$ per stage ($1/N$ overall)
Rounding	round-half-to-even + saturation
Complex multipliers	7 (last stage `HAS_TWIDDLE=0`)
Multiplier structure	3-real-multiply, 3-stage pipeline
Twiddle ROM	128×32 b, synchronous, 1-cycle
Window	Hann, 256×16 b ROM, Q1.15
Magnitude	α-max-β-min, $\alpha = 31471$, $\beta = 13036$
Log magnitude	Q8.8, optional (`ENABLE_LOG`)
Output	128 single-sided bins, natural order
Core latency	291 valid samples (constant)
Sample rate / resolution	48 kHz / 187.5 Hz per bin
Interface	AXI4-Stream, `sof/valid/last`
Target device	XC7Z020-1CLG400 (PYNQ-Z2)

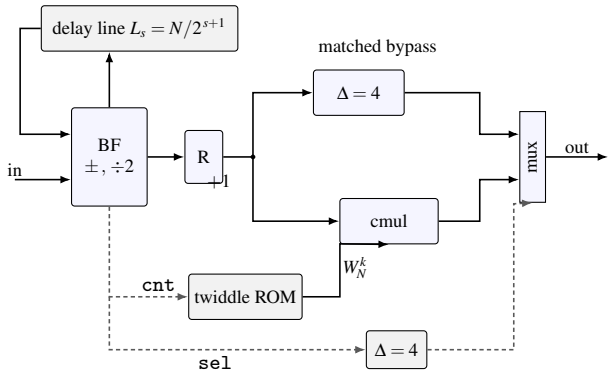

Fig. 2. One R2SDF stage. The feedback delay line contributes L_s valid samples of latency, the butterfly output register one, and the twiddle branch four; the sum path is delayed by a matched four samples so that the two multiplexer inputs are time-aligned. A one-sample mismatch between the branches, or between the commutator select and the data it labels, rotates the whole spectrum by a fixed twiddle factor (Section V C).

master interfaces [20] carrying `tdata`, `tvalid`, `tready` and framing bits, and is written in synthesisable Verilog-2005 [21] with a SystemVerilog system-level testbench [22]. All parameters — transform length, word length, latency and the presence of a twiddle multiplier in a given stage — are Verilog `parameters` so that a single source file serves every stage instance.

B. Input Buffering and Windowing

The input buffer FSM implements a ping-pong pair of 256-sample memories so that a frame can be windowed and transformed while the next frame is still arriving. It emits a start-of-frame pulse with the first sample of every frame. As a consequence of the ping-pong hand-over the FSM inserts exactly one idle cycle between consecutive frames; that single bubble is the reason for the timing discipline described in Section III D. The window unit multiplies each sample by the Q1.15 Hann coefficient read from ROM and rounds the Q2.30 product back to Q1.15. Both the window ROM and the twiddle ROM are generated by the same Python model that later serves as the verification oracle, which removes an entire class of coefficient mismatch by construction.

C. FFT Core

The core instantiates eight identical parameterised stages (Fig. 2). Stage s contains

- a feedback delay line of $L_s = N/2^{s+1}$ complex words, implemented as a shift register with clock enable so that the synthesiser can map long lines to block RAM and short ones to SRL/distributed RAM;
- a radix-2 butterfly producing $(a+b)/2$ and $(a-b)/2$ with the rounding and saturation rule of (6), followed by one output register;
- a twiddle branch consisting of the synchronous twiddle ROM (one cycle) and the three-stage complex multiplier (three cycles), and a bypass path for the sum stream delayed by exactly four valid samples to match it;
- a commutator select and sample counter that are *registered alongside the data* rather than tapped from the current counter state.

The last stage is instantiated with `HAS_TWIDDLE=0` because $W_2^0 = 1$; it therefore contains no multiplier, giving the seven multipliers

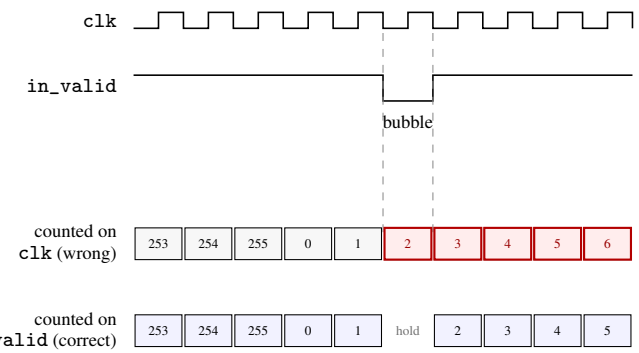

Fig. 3. Why the pipeline is gated on `valid`. The ping-pong buffer inserts one idle cycle between frames. A framing counter clocked unconditionally slips by one position per frame and the error accumulates; a counter clocked by `valid` holds through the bubble and stays aligned with the data it labels.

of Table I. The complex multiplier implements (8) with three DSP48E1 slices and a single round-and-saturate at the output.

D. Sample-Domain (Valid-Gated) Pipeline Discipline

Every register in the datapath, including the delay lines and the framing delay lines, is clock-enabled by the `valid` signal. The pipeline therefore advances in the *sample* domain, not the clock domain, and all latency figures in this paper are quoted in valid samples.

This choice has three consequences. First, the single inter-frame bubble produced by the ping-pong buffer is transparent: the pipeline simply does not advance during that cycle. Had the framing counters been clocked unconditionally, each frame would have drifted by one cycle relative to its data, and — because the frames are back to back — the error would have accumulated, corrupting the bit-reversal indices and hence the whole spectrum after a few frames. Second, the design tolerates an arbitrarily sparse input: a 48 kS/s microphone feeding a 50 MHz fabric delivers one sample every 1042 clocks, and no FIFO, rate matcher or handshake logic is needed for the core to remain correct. Third — and this is the price — the pipeline does not self-flush: `out_valid` is asserted only while `in_valid` is being asserted, so when the source stops, the last 291 samples remain resident. This is a documented behaviour rather than a defect (the testbench evaluates the third frame of a continuous stream), but a flush mechanism would be required for a burst-mode host, and is listed in Section VIII.

E. Latency Model and Frame Labelling

The framing outputs (`out_sof`, `out_valid`, `out_last`) are produced by delaying the corresponding input signals by a compile-time constant `LATENCY`, expressed in valid samples. Downstream blocks — in particular the bit-reversal buffer — use those labels to decide which bin a given sample belongs to. `LATENCY` is therefore not a documentation comment: it is a functional parameter, and a wrong value silently relabels the entire spectrum.

Table III decomposes the correct value. Three mechanisms contribute: the delay lines (5), the butterfly output register in every stage, and the twiddle branch in the seven stages that have

TABLE III
LATENCY DECOMPOSITION OF THE 256-POINT CORE

Contribution	Per stage	Stages	Valid samples
SDF feedback delay lines	$N/2^{s+1}$	8	$\sum L_s = 255$
Butterfly output register	1	8	8
Twiddle ROM	1	7	7
Complex multiplier pipeline	3	7	21
Total `LATENCY`			**291**
Original (incorrect) value			283

one. Only the first is visible in the delay-line parameter array, which is precisely why the original derivation, $255+28=283$, omitted the eight output registers.

F. Magnitude, Reordering, and Peak Detection

The magnitude unit computes (10) on the absolute values of the real and imaginary parts and, in parallel, a $\log_2$ approximation via a 16-level leading-one detector and mantissa interpolation. In its original single-cycle form this unit performs absolute value, comparison, two multiplications, a 33-bit addition, rounding, saturation, leading-one detection and interpolation between two register stages, and it is the critical path of the standalone IP (Section V E). A four-stage pipelined variant with identical ports has been written and verified bit-exact against the original over 200 000 vectors; it is timing-safe because the downstream blocks align themselves on `sof`/`last` rather than on a fixed cycle count.

The bin RAM writes the magnitude stream at bit-reversed addresses and reads out the 128 single-sided bins in natural order; the peak detector reduces each frame to a `peak_bin`/`peak_mag` pair with a `peak_valid` strobe.

G. Board-Level Self-Test Harness

The target board, a PYNQ-Z2 carrying an XC7Z020, has no PDM microphone, so the board-level wrapper contains an internal stimulus generator that replays a stored test vector corresponding to a tone at bin 10 (1875 Hz). The 125 MHz board oscillator is divided by a mixed-mode clock manager (MMCM) to 49.98 MHz for all internal logic, for the timing reasons given in Section V E; the reset is held until the MMCM asserts `locked`, and the stimulus generator's sample-period constant is rescaled accordingly. The acceptance criterion is deliberately trivial to observe: four LEDs display the low bits of `peak_bin`, so a correct system shows the pattern 1010_2 with the display enabled and 0000_2 with it disabled, while one RGB LED indicates `busy` and a second pulses on every `peak_valid`. An optional PDM front end (CIC decimator plus FIR compensator [17]) exists for a microphone-equipped board and is excluded from the build reported here.

IV. Verification Methodology

A. Bit-Accurate Reference Model

The oracle is a Python model that reproduces the RTL arithmetic exactly: the same Q1.15 representation, the same one-half scaling schedule, the same round-half-to-even and saturation rules,

TABLE IV
VERIFICATION MATRIX: ALL SEVEN LEVELS ARE BIT-EXACT AGAINST THE REFERENCE MODEL

#	Unit under test	Oracle and method	Stimulus volume	Result
1	`complex_multiplier` (3-multiply)	`cmul_q15()`, direct 4-multiply form	200 000 random vector pairs	bit-exact
2	`twiddle_rom`	independent `twiddle_q15()` + 2 analytic anchors	all 128 addresses (exhaustive)	bit-exact
3	`butterfly_r2_stage`	pure-butterfly model, twiddle path removed	1024 cycles (4 frames of $N = 256$)	bit-exact; `sel`/`cnt` aligned
4	`fft_r22sdf_top`	`fft_core_frame()`, full frame, offset sweep	frames $\times$ offsets ± 24	bit-exact; latency offset $= 0$
5	`magnitude_unit`	step-by-step translation of the RTL	2013 pairs (13 boundary + 2000 random)	bit-exact (magnitude and $\log_2$)
6	`bin_ram_output`	cycle-accurate model + visual check	1164 cycles; 128/128 bit-reversed samples	bit-exact
7	full system (`tb_top.sv`)	end-to-end, continuous stream	8 consecutive frames	`peak_bin`= 10, `peak_mag`= 5507

Auxiliary checks: `magnitude_unit_pipelined` vs. original: 200 000 vectors, bit-exact · hierarchical port audit: 119 connections, 0 width/direction mismatches
CIC integrator wraparound (optional PDM front end): 400 000 samples, bit-exact against unbounded-precision arithmetic

and the same twiddle and window coefficients — indeed the model *generates* the `.mem` files that initialise the ROMs, so a coefficient mismatch between model and hardware is structurally impossible. The model is written at the level of frames and values, not cycles; that division of labour is deliberate, and Section V B shows exactly where its limits lie. Its own floating-point path was first checked against `numpy.fft`, agreeing to $\approx 10^{-16}$, so that a discrepancy against the RTL can be attributed to the fixed-point emulation rather than to the transform itself.

B. Layered Verification

Verification proceeds bottom-up through seven levels (Table IV). Each level compares an RTL block against the corresponding model function over a stimulus set chosen to exercise the block's own failure modes: random vectors where the operator is combinational and its error surface is smooth, exhaustive enumeration where the state space is small enough (all 128 twiddle addresses), boundary values where saturation is possible, and cycle-by-cycle comparison where the block has internal state. The criterion at every level is bit-exactness, not a tolerance: a fixed-point design has a unique correct answer, and accepting "close enough" would forfeit the ability to localise a one-LSB rounding difference.

The value of the layering is diagnostic rather than statistical. A system-level test tells you that the spectrum is wrong; it does not tell you which of eleven modules is responsible. Levels 1–3 in particular were added after a phase-rotation defect (Section V C) had been diagnosed the slow way.

C. Self-Measuring Latency Testbench

Rather than assume the declared latency, the core testbench sweeps a frame-alignment offset over a window around the declared value, computes the signal-to-quantisation-noise ratio (SQNR) against the reference frame at every offset, and reports the offset that maximises it. A correct declaration yields a best offset of exactly zero. The original sweep window of ± 6 was, as it turned out, narrower than the error it was meant to catch ($+8$); it has been widened to ± 24 and now emits an explicit warning when the optimum lands on the window boundary, which is the signature of a sweep that is too narrow rather than of a design that is correct.

D. What the SQNR Numbers Mean

The core testbench reports 204.6 dB, which is not a measurement of anything physical. To avoid a division by zero the testbench clamps the noise energy at 10^{-12}; with the signal energy of the test frame, 2.875×10^8, a *zero-error* result therefore prints $10\log_{10}(2.875 \times 10^8 / 10^{-12}) = 204.6$ dB. The number should be read as a flag meaning "no mismatch", and the honest statement of the result is the one used throughout this paper: the RTL is bit-exact with the reference model. For calibration, injecting a single LSB of error into a single bin drops the same metric to 84.6 dB, so any value below about 90 dB in this testbench indicates a real arithmetic difference.

The physically meaningful accuracy figure is a different comparison: the fixed-point core against a double-precision FFT of the same input. That comparison yields 58–61.6 dB across random seeds and is bounded by the Q1.15 format, not by the implementation.

E. Independent Re-Derivation

To confirm the accuracy claims without reusing the project's own model, a second bit-accurate model was written from the specification alone for this paper and exercised as follows.

1) Three-multiply identity: 200 000 random Q1.15 vector pairs produced 0 mismatches out of 400 000 compared real and imaginary results against the four-multiply form, confirming level 1 of Table IV.

2) Word-length sweep: Fig. 4 plots the core SQNR against a double-precision reference for $B \in \{10, \ldots, 20\}$ bits, at eight random seeds per point. The measured slope is 6.02 dB/bit, exactly the theoretical 1 bit $\rightarrow$ 6.02 dB relation, and the implemented Q1.15 point gives 58.97 ± 0.41 dB, in agreement with the project's own ≈ 60 dB measurement. The first-order bound (7) predicts 68.0 dB; the 9 dB shortfall is the combined contribution of twiddle-coefficient quantisation and of the rounding inside each of the seven complex multipliers, neither of which (7) accounts for.

3) Self-test stimulus: feeding the model with the Hann-windowed bin-10 tone used by the on-board self test reproduces the hardware result — peak bin 10, i.e. 1875 Hz — and shows the expected -6.02 dB neighbours (Fig. 5). The α-max-β-min stage adds a peak error of 3.96 % with the equiripple signature of Fig. 6b, matching the design coefficients exactly.

F. Structural Audit and Tool Diversity

Two checks complement the functional levels. A script walks the design hierarchy and compares the width and direction of every port against every connection made to it: 119 connec-

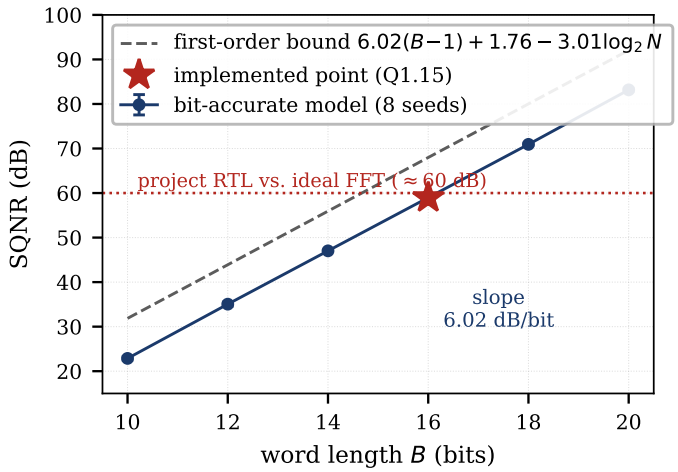

Fig. 4. SQNR of the fixed-point core against a double-precision reference, as a function of data word length, from the independent bit-accurate model (eight seeds per point, error bars ±1 standard deviation). The slope matches the 6.02 dB/bit theoretical rate; the constant offset below the first-order bound (7) is due to twiddle quantisation and multiplier rounding.

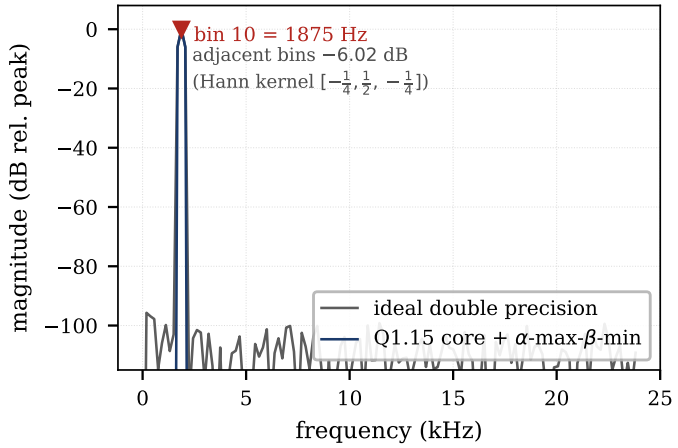

Fig. 5. Single-sided spectrum of the on-board self-test stimulus (Hann window, tone on bin 10) computed by the bit-accurate model of the implemented datapath, against a double-precision reference. The fixed-point curve tracks the reference down to the Q1.15 quantisation floor near −80 dB.

tions, zero mismatches. Separately, the design is elaborated by more than one tool. One defect — a part-select `in_sof[0 +: USER_WIDTH]` applied to a scalar — was accepted silently by an open-source simulator and rejected by the vendor synthesiser as `[Synth 8-373] invalid index for scalar`. Passing simulation is therefore not evidence that a construct is synthesisable, and running both tool chains costs little.

A third, benign finding is recorded for completeness: the output register `rd_data` in the bin RAM is not explicitly cleared in the reset branch, so it is undefined for one to two cycles after reset. It is harmless because `out_valid` is low throughout that interval, and it is noted here only because a verification flow that reports such items is more trustworthy than one that reports none.

TABLE V
EFFECT OF THE LATENCY CONSTANT ON SYSTEM BEHAVIOUR

Check	`LATENCY`= 283	`LATENCY`= 291
Core TB, offset sweep	SQNR −3.0 dB peak bin 122 (fail)	SQNR 204.6 dB† peak bin 10 (pass)
System TB, 8 frames	`peak_bin`= 26 (fail)	`peak_bin`= 10 (pass)
Arithmetic blocks	bit-exact	bit-exact

†Clamp-limited value denoting zero error; see Section D.

V. Results and Key Findings

A. Functional Correctness

All seven verification levels pass bit-exactly (Table IV). At system level, eight consecutive frames of the internal self-test stimulus produce an identical `peak_bin`= 10 and `peak_mag`= 5507, confirming that the framing is not merely correct once but stable across frame boundaries — the property that the defect described next destroyed.

B. Key Finding 1: A Framing Error Invisible to the Golden Model

The most instructive defect in the project was not an arithmetic error. Every arithmetic block was already bit-exact when the complete system reported the wrong spectrum: the core testbench found a peak at bin 122 instead of 10 and an SQNR of −3.0 dB, and the system testbench reported `peak_bin`= 26 on every frame (Table V).

The cause was the latency constant. `LATENCY` had been derived on paper as the sum of the delay lines plus the twiddle-branch pipelines, $255 + 28 = 283$, which silently omits the one output register that each butterfly stage holds outside its delay-line array — eight registers, eight valid samples. Because the framing signals are generated by delaying `in_sof` by `LATENCY`, every bin label was displaced by eight positions, and after bit-reversal an eight-position displacement maps to an essentially arbitrary bin. The datapath was right; the clock tick at which its output was declared to be bin k was wrong.

Three independent lines of evidence converge on 291: the corrected analytic decomposition of Table III; the measured optimum of the offset sweep in HDL simulation; and the restoration of exact agreement at system level. The reference model itself could not have found the error, and this is the general point. The model computes *values* for a frame; it does not model the cycle at which each value emerges from a pipeline. A software oracle validates arithmetic. It cannot validate time, and in a streaming design time is half of the specification.

Two secondary observations reinforce the lesson. First, the offset sweep that was supposed to catch exactly this class of error searched ±6 around the declared value while the error was +8: a verification parameter chosen from the same assumption as the design parameter it checks provides no independent evidence. Second, the constant appeared in seven files — the FFT core, three integration wrappers, two board wrappers and two testbenches — so a partial correction would have produced a design that was internally inconsistent rather than merely wrong.

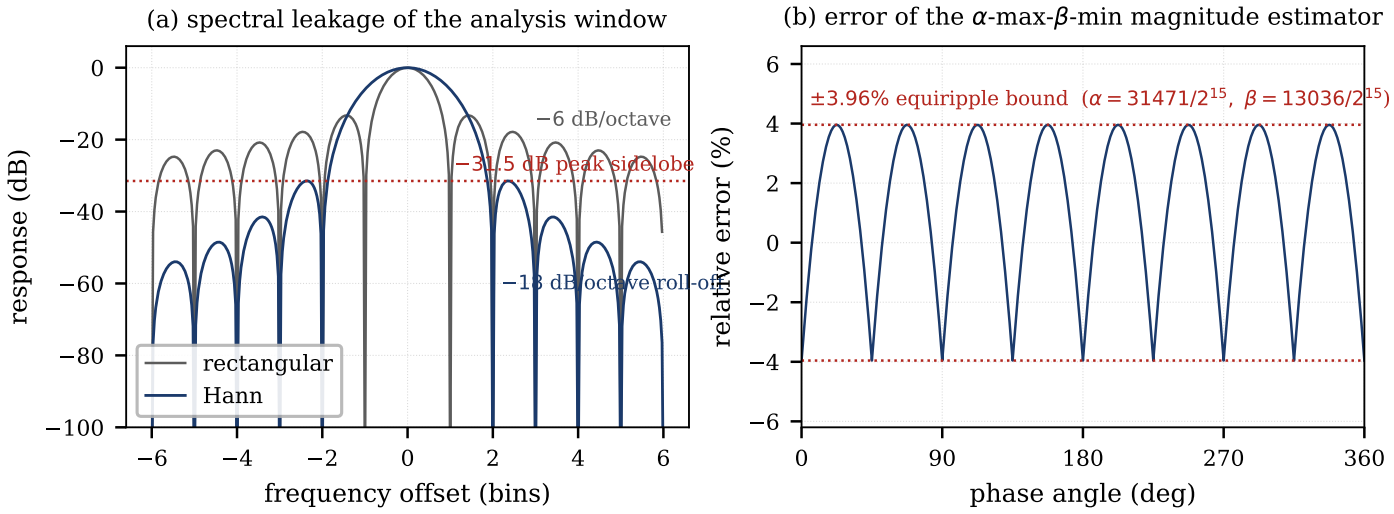

Fig. 6. (a) Spectral leakage of the $N = 256$ Hann window against the rectangular window: -31.5 dB peak sidelobe and -18 dB/octave roll-off against -13.3 dB and -6 dB/octave, at the cost of a main lobe twice as wide (ENBW 1.5 bins) [14]. (b) Relative error of the implemented α-max-β-min magnitude estimator with the quantised coefficients $\alpha = 31471/2^{15}$, $\beta = 13036/2^{15}$; the error is equiripple within $\pm 3.96\,\%$ and depends only on phase angle.

Both are addressed by rule R3 and R6 in Section VII.

C. Key Finding 2: The Phase-Rotation Fingerprint

An earlier defect in the same family produced a distinctive symptom: the entire spectrum was rotated in phase by exactly W_8^1. The cause was that the commutator select and sample counter, `sel_o` and `cnt_o`, were driven combinationally from the registers that had *already* been updated for the next sample, so the control labels ran one cycle ahead of the data they described. The twiddle factor applied to each sample was consequently the neighbouring one, which is a constant multiplicative rotation — and a constant rotation is a far more informative symptom than random corruption. A spectrum that is uniformly rotated by a single twiddle factor is almost always a control/data skew of exactly one sample; a spectrum whose peak has moved to an unrelated bin is almost always a framing offset. Catalogu-ing symptoms this way converted subsequent debugging from bisection into recognition.

D. Key Finding 3: Fixed-Point Accuracy

The accuracy results of Section E are summarised as follows. The implementation is bit-exact with its reference model; against an ideal transform it achieves 58.97 dB SQNR at Q1.15, tracks the 6.02 dB/bit slope over a 10–20 bit sweep (Fig. 4), and re-produces the expected Hann kernel and the 3.96 % equiripple magnitude error (Figs. 5, 6). For a spectrum analyzer whose output drives a peak detector and a decibel display, 59 dB of dynamic range is comfortable: it exceeds the ≈ 48 dB an 8-bit display could resolve and the typical ≈ 60 dB working range of an electret microphone front end, though it would be marginal for a measurement instrument, where the sweep of Fig. 4 indicates that $B = 18$ (about 71 dB) would be the next sensible point.

E. Key Finding 4: Timing Behaviour Depends on the Constraint

Three implementation runs were performed (Table VI). Taken together they show why a single worst-negative-slack (WNS) number is not a portable statement about a design.

Constrained at 100 MHz out of context, the IP misses timing by 5.590 ns on 387 of 10 423 endpoints, implying a critical path of 15.590 ns and $F_{\max} \approx 64.1$ MHz. Driven directly from the 125 MHz board oscillator the same logic misses by 7.936 ns, implying 15.936 ns and 62.7 MHz — consistent, as it should be, since the logic is unchanged. Inserting an MMCM to divide the board clock to 49.98 MHz closes timing comfortably: WNS $= +2.510$ ns, total negative slack 0, no failing endpoints, and no critical warnings from either synthesis or implementation once a duplicate `create_clock` declaration between the board constraint file and the MMCM's generated constraints was removed.

The third row also carries a methodological warning. Its positive slack implies a critical path of 17.498 ns, i.e. an apparent $F_{\max}$ of 57.2 MHz — *lower* than the two failing runs. Nothing became slower: place-and-route stops optimising once the constraint is met, so the achieved path length under a loose constraint is an upper bound on delay, not a measurement of the design's capability. $F_{\max}$ must therefore be quoted from a run whose constraint actually binds, which is why this paper reports ≈ 64 MHz from the 100 MHz run and not 57.2 MHz from the run that passes. A second reason to distrust the third row as a capability measure is that it implements a larger design: the board build carries the MMCM, stimulus generator and status logic of Table VII, whereas rows one and two characterise paths through the analyzer itself.

The reported critical paths are worth recording because they differ between the two configurations. Out of context, the bottleneck is the single-cycle magnitude unit, which performs absolute value, comparison, two multiplications, a 33-bit addition, rounding, saturation and a 16-level leading-one detection between two registers. In the board configuration the worst path is reported

TABLE VI
TIMING RESULTS UNDER THREE CONSTRAINT SCENARIOS (XC7Z020-1)

Run configuration	Clock (MHz)	T (ns)	WNS (ns)	Path (ns)	Implied F_{max}
IP out of context	100	10.000	−5.590	15.590	64.1 MHz
Board top, direct	125	8.000	−7.936	15.936	62.7 MHz
Board top, MMCM	49.98	20.008	+2.510	17.498	(57.2 MHz)‡

‡Not a capability measurement: the constraint is not binding, so the tool stops optimising and the achieved path length overstates the true delay. Only the first two rows are admissible as F_{max} estimates. The third row is the shipping configuration: TNS = 0.000 ns, 0 failing endpoints, all constraints met. An earlier implementation run of the same configuration reported WNS = +1.732 ns; the difference is run-to-run placement variation, not a design change, and both runs meet all constraints.

TABLE VII
RESOURCE UTILISATION ON XC7Z020-1CLG400: COMPLETE BOARD BUILD VERSUS THE IP CHARACTERISED OUT OF CONTEXT

Resource (device total)	Board build*	IP out of context†
Slice LUT (53 200)	4432 (8.33 %)	≈3192 (6 %)
as distributed RAM (17 400)	—	≈870 (5 %)
Slice register (106 400)	4328 (4.07 %)	≈2128 (2 %)
Occupied slice (13 300)	1882 (14.15 %)	—
Block RAM tile (140)	7.5 (5.36 %)	≈4 (3 %)
DSP48E1 (220)	**24** (10.91 %)	**24** (11 %)
Bonded IOB (125)	—	≈99 (79 %)

*Complete `pynq_z2_top` build after place and route, including the MMCM, reset synchroniser, self-test stimulus generator and LED logic; counts are read directly from the utilisation report. †IP alone under `timing_only.xdc`, which assigns no pins; the tool reports utilisation percentages and the absolute counts are derived from the device totals [24], so they carry a rounding uncertainty of up to half a percentage point. The 79 % IOB figure is an artefact of out-of-context characterisation, which promotes every port to a package pin; in the integrated design the same ports are internal AXI4-Stream connections. Dashes mark quantities not reported for that configuration.

between the input buffer FSM and the window unit, roughly 20 logic levels through a DSP48E1. The pipelined magnitude unit described in Section III F addresses the first; the second has not yet been re-characterised, and both are open items (Section VIII).

F. Hardware Status

The design synthesises and implements cleanly, meets all timing constraints at 49.98 MHz, and the resulting bitstream has been programmed onto the XC7Z020. The acceptance procedure of Section III G passes on silicon: with the self-test source selected the LEDs display 1010_2, i.e. the peak bin is 10, and 0000_2 with it disabled. The complete signal chain — windowing, transform, magnitude, bit-reversal, framing and peak detection — therefore runs correctly on the device and not only in simulation.

Two qualifications keep this claim inside its evidence. The confirmation is of a scalar, the peak bin index, displayed on four LEDs; the 128-bin spectrum has not been read back over AXI4-Stream and compared against the reference model on hardware, so bit-exactness on silicon is inferred from the peak rather than demonstrated bin by bin. And every other result in this paper — utilisation, timing, power — remains a tool report, not a measurement on the device. We state this explicitly because the distinction between "implemented", "validated on hardware" and "characterised on hardware" is routinely blurred.

VI. PPA CHARACTERIZATION

A. Area

Table VII reports post-implementation utilisation on the XC7Z020 for two distinct configurations, because conflating them is a common way to misstate the area of an IP core.

The *board build* is the complete `pynq_z2_top` design that produces the shipping bitstream. Besides the analyzer it contains the MMCM, the reset synchroniser, the self-test stimulus generator with its own vector ROM, and the LED status logic. The *out-of-context* run implements the IP alone under a timing-only constraint file that assigns no pins; it is the configuration in which the core's own cost is visible, and its 79 % IOB figure is an artefact of that choice rather than a property of the design, since every port is promoted to a package pin when there is nothing to connect it to.

The difference between the two columns is therefore the wrapper, and it behaves as expected. Logic and registers grow by roughly 1200 and 2200 respectively, block RAM by about three and a half tiles — the stimulus ROM accounts for most of that — and the DSP count does not move at all, because none of the added blocks contains a multiplier. That last invariance is a useful check in itself. Either way the design is small: even the full build leaves 91 % of the LUTs, 89 % of the DSP slices and the entire processing system free for a host interface, a DMA engine or a downstream classifier.

The DSP figure admits an exact analytic prediction. Each complex multiplier uses three DSP48E1 slices by (8), the magnitude unit uses two (one for α, one for β), and the window unit uses one:

$$N_{\mathrm{DSP}} = 3\,(\log_2 N - 1) + 2 + 1 = 3 \times 7 + 3 = 24, \qquad (12)$$

which matches the tool report in both configurations (24 of 220 slices, 10.91 %). An analytic model that predicts a measured resource count exactly is a useful cross-check on both: it confirms that no multiplier was inferred where the designer did not intend one, and that none was optimised away into logic.

Storage admits the same treatment. The delay lines hold $N-1 = 255$ complex words of 32 b (8160 b), the twiddle ROM 128×32 b (4096 b), the window ROM 256×16 b (4096 b) and the output bin RAM 128×16 b (2048 b), i.e. 18 400 b in total. The tool allocates roughly four 36 Kb block RAM tiles (≈ 151 Kb) plus 870 LUTs of distributed RAM to the core alone, a storage efficiency of about 12 %; the 7.5 tiles of the board build include the self-test stimulus ROM, which is not part of the IP. This is expected rather than wasteful: a 128×32 b delay line occupies a whole tile whatever its depth, and the shallow lines of the later stages ($L_s \leq 16$) map to shift-register-LUT or distributed RAM instead. It is nevertheless the second lever, after the multiplier count, for a smaller implementation.

B. Performance

Table VIII collects the performance figures. The architecture accepts one sample per enabled clock cycle, so at $F_{max} \approx 64$ MHz

TABLE VIII
PERFORMANCE SUMMARY

Metric	Value
Architectural throughput	1 sample per enabled clock cycle
$F_{\max}$ (binding constraint)	≈ 64 MHz
Max sustained sample rate	64 MS/s
Equivalent transform rate	250 000 transforms/s
Equivalent operation rate	2.56 GFLOP/s ($5N\log_2 N$)
Required for 48 kHz audio	48 kS/s ($\approx 1330\times$ margin)
Shipping clock	49.98 MHz, WNS +2.510 ns
margin at shipping clock	$\approx 1040\times$
Core latency	291 valid samples
at 48 kS/s	6.06 ms
at 64 MS/s	4.55 μs
Frame period / update rate	5.33 ms / 187.5 spectra s^{-1}
Frequency resolution	187.5 Hz, 128 single-sided bins
Total on-chip power	0.259 W (vectorless, low confidence)

the core sustains 64 MS/s, equivalently 250 000 transforms per second or 2.56 GFLOP/s using the conventional $5N\log_2 N$ operation count. The application needs 48 kS/s. The margin is a factor of about 1330, and it is the single most important number in this table, because it says that every architectural decision in the design was optimised for the wrong constraint: throughput was never scarce.

The latency deserves a precise statement because it is expressed in the sample domain. The core emits the first bin of a frame 291 valid samples after that frame's first input sample. At 48 kS/s this is 6.06 ms — longer than the 5.33 ms frame period, which is normal for a pipelined SDF and simply means that consecutive frames overlap inside the pipeline. If the same core is fed at full rate from a buffer, the latency is 4.55 μs.

C. Power, and Why We Do Not Report Energy per Transform

Vivado reports a total on-chip power of 0.259 W with the confidence level marked *Low*, because the estimate is vectorless: no switching activity file was supplied, so node activity is inferred from default assumptions rather than from simulated toggling [26]. Two consequences follow, and both are reasons to treat the figure as provisional.

First, at this utilisation a large fraction of the total is device static power, which is a property of the XC7Z020 and its junction temperature, not of the core; the dynamic power actually attributable to the FFT cannot be separated out of a single total. Second, dividing the total by the 187.5 spectra/s update rate would give 1.4 mJ per transform, a figure three to four orders of magnitude larger than any plausible switching energy for 10^4 arithmetic operations in 28 nm logic — which is exactly what one should expect when a mostly-static total is attributed to a workload that keeps the fabric idle 99.9 % of the time. We therefore report the total as measured, and decline to derive an energy-efficiency metric from it. Obtaining a defensible number requires a post-implementation timing simulation, a switching activity interchange format (SAIF) file, and a power report driven by that file; this is the first item of future work.

D. The Cost of the Architectural Choice

The design uses plain R2SDF, in which every stage but the last carries a general complex multiplier. The accepted baseline for this transform size is R2^{2}SDF [7], which restructures the same graph so that half of the rotations become multiplications by $-j$ — a swap and a sign change — while keeping the radix-2 butterfly and the $N-1$ memory. Applying (12) to both:

$$\begin{aligned} \text{R2SDF:} &\quad 3(\log_2 N - 1) + 3 = 24 \text{ DSP48E1}, \\ \text{R2}^2\text{SDF:} &\quad 3(\log_4 N - 1) + 3 = 12 \text{ DSP48E1}. \end{aligned} \tag{13}$$

The choice therefore costs 12 DSP slices, half of the design's multiplier budget, for no benefit in memory, adder count or control complexity. We report this rather than omit it: it is the single highest-leverage change available to the design, and any comparison against published cores must be read in that light.

A second, larger inefficiency follows from Table VIII. With a $1330\times$ throughput margin, a streaming architecture is not merely suboptimal but categorically the wrong family for this workload. A memory-based (iterative) FFT that reuses a single butterfly and a single multiplier needs $\frac{N}{2}\log_2 N = 1024$ butterfly operations per frame, i.e. four clock cycles per input sample, and would therefore run comfortably at under 200 kHz — at roughly one eighth of the multiplier count and with the delay lines replaced by a single dual-port memory. Third, because the input is real, half of the datapath computes with a zero imaginary part throughout; the standard real-input factorisation transforms a length-N real sequence with an $N/2$-point complex FFT plus $\mathcal{O}(N)$ post-processing [2], halving arithmetic and memory again. Each of these is a straightforward, well-documented transformation, and together they bound how far the present implementation sits from the efficient frontier.

E. Threats to Validity

The PPA numbers above are tool reports from a single device, a single tool version and a single set of constraints; no vendor or academic baseline was implemented under identical conditions, so absolute comparisons with published cores are not supported by this data. The two columns of Table VII must not be mixed: the board-build counts are read directly from the utilisation report, whereas the out-of-context counts are derived from reported percentages and inherit a rounding uncertainty of up to half a percentage point of the device total. Quoting the smaller column as the cost of the delivered system, or the larger as the cost of the IP, would misstate the design in opposite directions. The power figure is vectorless. The timing figures are static-analysis results, and the run-to-run spread of 0.78 ns observed on the shipping configuration is a reminder that a single WNS value carries at least that much uncertainty. We regard Table VII and the first two rows of Table VI as reliable, Table VIII as reliable except for the power row, and any cross-design comparison as future work (Section VIII).

VII. Lessons Learned as Design Rules

Each rule below is stated in general form and followed by the specific incident that produced it. They are not novel individually — most are folklore — but the incidents give them concrete cost, and the same failures recur in student and production designs alike.

R1 — A control label must be registered in step with the data it labels. The commutator select and counter were driven from registers already updated for the next sample and therefore led the data by one cycle, applying each sample's neighbouring twiddle factor. Symptom: a uniform W_8^1 phase rotation of the whole spectrum.

R2 — Delay-match parallel branches by construction, not by inspection. The twiddle branch (ROM + multiplier = 4) and the bypass ($\Delta = 4$) must be equal. If the multiplier pipeline depth changes from three to P, the bypass must become $1 + P$ *and* the core latency changes with it; deriving both from one local parameter makes the dependency impossible to forget.

R3 — Latency must be measured by the testbench, never asserted by the designer. The declared 283 was wrong by eight samples and no arithmetic test could see it. The offset-sweep testbench that was supposed to catch it searched only ± 6 — a verification window derived from the same belief as the design parameter it was checking. Sweep windows must exceed the plausible error and must warn when the optimum lands on a boundary.

R4 — A value-level reference model validates arithmetic, not time. This division of labour should be written down explicitly, because the reference model's own structural simplification — treating a stage as an immediate function of its predecessor, thereby flattening eight inter-stage registers — is invisible from inside the model. Pipeline timing is verifiable only against an HDL simulation.

R5 — Quote $F_{\max}$ only from a binding constraint. A run with positive slack reports the delay the tool stopped at, not the delay the design is capable of; here the passing run understated $F_{\max}$ by 15 % relative to the failing runs (Table VI).

R6 — Give every cross-cutting constant a single source of truth. The latency constant appeared in seven files, and one testbench printed a hard-coded value while instantiating a parameterised one, so the report could disagree with the design under test. Instantiation and reporting must read the same symbol.

R7 — Gate the pipeline on `valid`, not on the clock, whenever the data rate is decoupled from the clock. One idle cycle per frame was enough to corrupt the spectrum cumulatively when framing counters ran on the clock. Gating on `valid` fixed it and, as a by-product, made the core tolerant of arbitrarily sparse sources — which is what allows a 48 kHz microphone to drive a 50 MHz pipeline with no rate-matching logic at all.

R8 — Elaborate with more than one tool. A part-select applied to a scalar passed an open-source simulator and was rejected by the vendor synthesiser. Simulation success is not evidence of synthesisability.

VIII. Limitations and Future Work

The following items are open, in the order in which they should be addressed.

Measurement completeness. The power figure must be regenerated from a post-implementation timing simulation with a SAIF file before any energy claim is made. Timing at 100 MHz should be re-run with the pipelined magnitude unit — already written and verified bit-exact — and the second reported bottleneck, the path between the input buffer and the window unit, characterised. Both results should be kept as an ablation: "before pipelining ($F_{\max}$, FF)" against "after" is a legitimate and informative pair, not a failure to be discarded.

Comparison. No baseline was implemented. A credible PPA claim needs the vendor FFT core [25] configured with the same N, word length and device, at minimum, plus at least one published academic core, and a sweep over word length $\{12, 14, 16, 18\}$, transform length $\{256, 512, 1024\}$ and clock frequency to place the design on a Pareto front rather than at a point.

Architecture. Section D identifies three transformations — R2^{2}SDF rotation folding, a memory-based rather than streaming datapath, and real-input factorisation — whose combined effect on this workload would be substantial.

Interfacing. The pipeline does not self-flush, so a burst-mode host would strand the last 291 samples; and the output stream is self-timed, so an AXI4-Stream FIFO is required ahead of a DMA engine that can apply sustained backpressure.

Hardware validation. The board-level acceptance test passes on silicon, but it confirms a single scalar — the peak bin — rather than the spectrum. Reading all 128 bins back over AXI4-Stream and comparing them against the reference model on the device would upgrade the claim from "the chain runs" to "the chain is bit-exact in hardware". The pin constraints have also not been checked against the official board master file.

Research directions. Two extensions would give the platform a question to answer rather than an artefact to exhibit: co-designing spectral front-end precision against a downstream task metric — keyword spotting accuracy, say, rather than SQNR — using the word-length sweep of Fig. 4 as the starting point; and an energy-per-detected-peak comparison between full FFT, Goertzel filter banks and sparse transforms in the small-N, always-on regime, where published sparse-FFT hardware is scarce because the literature has concentrated on very large transforms.

IX. Conclusion

We have presented a 256-point R2SDF FFT audio spectrum analyzer IP core in Q1.15 arithmetic, verified bit-exactly against a bit-accurate reference model at seven levels of its hierarchy and characterised for power, performance and area on an XC7Z020. The complete board build occupies 8.33 % of the device's LUTs and 24 DSP48E1 slices — exactly the number predicted by an analytic model, and unchanged from the out-of-context characterisation of the IP alone, which occupies 6 %. The core reaches $F_{\max} \approx 64$ MHz, which exceeds the 48 kS/s requirement by a

factor of 1330, and closes timing with +2.510 ns of slack at 49.98 MHz on the target board, where the peak-bin acceptance test passes on silicon. Its fixed-point accuracy, measured independently for this paper, is 58.97 dB SQNR at Q1.15 with the theoretical 6.02 dB/bit slope.

The result we consider most transferable is negative. A datapath can be bit-exact against its reference model at every level and still produce a completely wrong spectrum, because a value-level model validates arithmetic and not pipeline time. The eight-sample framing error documented here — one output register per stage, omitted from a latency constant derived on paper — moved the reported peak from bin 10 to bin 122 while leaving every arithmetic block provably correct. The remedy is cheap and general: express latency in the domain in which the pipeline actually advances, decompose it analytically, and then have the testbench measure it rather than believe it.

We also report, rather than omit, what the design costs and what it still lacks: an R2SDF datapath uses twice the multipliers of the accepted R2^2SDF baseline, the streaming family is three orders of magnitude over-provisioned for audio rates, the power figure is not yet vector-based, and hardware confirmation extends to the peak bin but not yet to the full spectrum. Stating these explicitly is what makes the remaining numbers usable.

References

[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, Apr. 1965.

[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[3] P. Duhamel and M. Vetterli, "Fast Fourier transforms: A tutorial review and a state of the art," *Signal Process.*, vol. 19, no. 4, pp. 259–299, Apr. 1990.

[4] L. R. Rabiner and B. Gold, *Theory and Application of Digital Signal Processing*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1975.

[5] E. H. Wold and A. M. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *IEEE Trans. Comput.*, vol. C-33, no. 5, pp. 414–426, May 1984.

[6] G. Bi and E. V. Jones, "A pipelined FFT processor for word-sequential data," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 37, no. 12, pp. 1982–1985, Dec. 1989.

[7] S. He and M. Torkelson, "A new approach to pipeline FFT processor," in *Proc. 10th Int. Parallel Process. Symp. (IPPS)*, Honolulu, HI, USA, Apr. 1996, pp. 766–770.

[8] M. Garrido, J. Grajal, M. A. Sánchez, and O. Gustafsson, "Pipelined radix-2^k feedforward FFT architectures," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 21, no. 1, pp. 23–32, Jan. 2013.

[9] M. Garrido, "A survey on pipelined FFT hardware architectures," *J. Signal Process. Syst.*, vol. 94, no. 11, pp. 1345–1364, Nov. 2022.

[10] P. D. Welch, "A fixed-point fast Fourier transform error analysis," *IEEE Trans. Audio Electroacoust.*, vol. AU-17, no. 2, pp. 151–157, Jun. 1969.

[11] T. Thong and B. Liu, "Fixed-point fast Fourier transform error analysis," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-24, no. 6, pp. 563–573, Dec. 1976.

[12] A. Karatsuba and Y. Ofman, "Multiplication of many-digital numbers by automatic computers," *Dokl. Akad. Nauk SSSR*, vol. 145, no. 2, pp. 293–294, 1962.

[13] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[14] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.

[15] A. E. Filip, "Linear approximations to $\sqrt{x^2+y^2}$ having equiripple error characteristics," *IEEE Trans. Audio Electroacoust.*, vol. AU-21, no. 6, pp. 554–556, Dec. 1973.

[16] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2010.

[17] E. B. Hogenauer, "An economical class of digital filters for decimation and interpolation," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-29, no. 2, pp. 155–162, Apr. 1981.

[18] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. New York, NY, USA: Wiley, 1999.

[19] J. Bergeron, *Writing Testbenches: Functional Verification of HDL Models*, 2nd ed. Boston, MA, USA: Kluwer Academic, 2003.

[20] Arm Ltd., "AMBA 4 AXI4-Stream protocol specification," Document ARM IHI 0051A, 2010.

[21] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005, 2006.

[22] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[23] Xilinx, Inc., "7 series DSP48E1 slice user guide," UG479, 2018.

[24] Xilinx, Inc., "Zynq-7000 SoC data sheet: Overview," DS190, 2018.

[25] Xilinx, Inc., "Fast Fourier transform LogiCORE IP product guide," PG109, 2022.

[26] Xilinx, Inc., "Vivado design suite user guide: Power analysis and optimization," UG907, 2021.

[27] Xilinx, Inc., "Vivado design suite user guide: Design analysis and closure techniques," UG906, 2021.

[28] TUL Corp., "TUL PYNQ-Z2 board reference manual," v1.0, 2018.